

「复制 MD」与「清空」按钮：是否影响整段对话？

Zotero AI Sidebar

2026-05-14

「复制 MD」与「清空」按钮：是否影响整段对话？

直接答案

按钮	是否修改本地状态	是否调用模型 API	对 “下一次发给模型的 payload” 的影响
复制 MD	否	否	零影响 —— 它只读不写，是导出动作
清空	是（彻底）	否	整段历史归零 —— 下一轮 thread / ledger / 滑窗回放全部从空白起算

一句话：两个按钮里，只有「清空」会影响后续对话；「复制 MD」是纯导出。

1. 「复制 MD」按钮的实现

源码位置：src/modules/sidebar.ts:630-662。

```
const copyAll = buttonEl(doc, " 复制 MD");
copyAll.title = state.copyDebugContext
  ? " 复制当前对话为 Markdown（含工具上下文和 PDF 片段）"
  : " 复制当前对话为 Markdown（只含论文介绍和对话）";
copyAll.addEventListener("click", () => {
  void (async () => {
    let systemPrompt: string | undefined;
    if (state.copyDebugContext) {
      // 仅在调试开关打开时构建 system prompt（含 ledger + 工具说明书）
      const ledger = formatContextLedger(state.messages);
      const built = await buildSystemContextOnly(state.itemID, ledger);
      systemPrompt = built.systemPrompt;
    }
    const markdown = formatConversationMarkdown(
      state,
      state.copyDebugContext,
      systemPrompt,
    );
    await copyToClipboard(doc, markdown, undefined, markdownToClipboardHTML(doc, markdown));
    flashButton(copyAll, " 已复制");
  })();
});
```

这个按钮做了什么

- 1. 读 state.messages（当前 Zotero item 的整段聊天）；

2. 调用 `formatConversationMarkdown` 把它格式化成 Markdown 字符串；
3. 写入剪贴板（纯文本 + HTML 两种格式，方便粘进富文本编辑器）；
4. 按钮文字临时变成“已复制”作为视觉反馈。

关于“调试上下文”开关 (`copyDebugContext`)

底部工具栏有一个开关：

- **关闭（默认）**：导出只包含论文介绍 + user/assistant 对话文本。
- **打开**：额外把 system prompt（含 ledger）、工具调用 trace、PDF 片段、思考内容（thinking）一并写入剪贴板。**这只影响导出格式，不影响后续 API 请求。**

它不做什么

- 不修改 `state.messages`；
- 不写入 `chat-history.json` 持久化文件；
- 不发起任何网络请求；
- 不更改 `state.copyDebugContext` 之外的任何 UI 状态。

对“对话”的影响

零。复制完之后再发下一条消息，wire payload 与点击按钮前完全一致——同样的 `[...history, userMessage]`、同样的 system prompt 拼装、同样的 ledger。复制 MD 只是把“模型现在看得到什么”做了一份人类可读的快照。

唯一一种“间接影响”是：你看了导出的内容之后，**自己**决定要不要点「清空」、要不要换一句话问、要不要修改之前哪条消息。这是人在改对话，不是按钮在改对话。

2. 「清空」按钮的实现

源码位置：src/modules/sidebar.ts:664-672。

```
const clear = buttonEl(doc, " 清空");
clear.disabled = state.sending;
clear.title = " 清空并保存当前条目的聊天记录";
clear.addEventListener("click", () => {
  state.messages = [];
  void saveChatMessages(state.itemID, state.messages);
  renderPanel(mount, state);
});
```

短短四行，但**每一行都对后续对话有结构性影响**。

逐行拆解

`clear.disabled = state.sending`

正在流式输出（`state.sending === true`）时按钮置灰。原因：清空 `state.messages` 会让正在进行的 tool loop 找不到自己应该写入的 assistant 槽，造成 UI 错乱与悬空 promise。

`state.messages = []`

直接把整段内存里的对话清零。这一行决定了下一次发请求时 `[...history, userMessage]` (sidebar.ts:4034) 的 history 部分为空数组——也就是说：

- 历史 user/assistant 文本：全没了；

- 历史 function_call / function_call_output: 全没了;
- 滑窗回放选区 / 标注 / PDF 检索片段: 全没了 (没有历史轮可供回放);
- ledger: formatContextLedger([]) 返回字符串 "none", 也就是说 system prompt 后缀变成 “之前没有任何上下文”;
- 保留的 “最近 4 轮 / 8000 字” 预算: 自然没有内容可填。

`saveChatMessages(state.itemID, [])`

调用 `src/settings/chat-history.ts:124-141`。关键代码:

```
if (safeMessages.length === 0) {
  delete threads[key]; // 整条 thread 从磁盘 JSON 里抹除
} else { ... }
await writeThreads(threads);
```

注意这不是 “把消息数组写成空数组”，而是**把整个 thread 条目从 `zotero-ai-sidebar-chat-history.json` 里删掉**。即使你之后切到别的 item 再切回来，这个 item 的对话历史也回不来——除非你之前从云端同步过备份。

`renderPanel(mount, state)`

重绘 UI。因为 `state.messages.length === 0`，「复制 MD」和「清空」按钮本身也会消失（它们的渲染条件是 `state.messages.length > 0`，见 `sidebar.ts:629`）。

对“对话”的影响

毁灭性。这是这个 UI 里**唯一会主动丢数据的常规按钮**。

清空之后下一轮的 wire shape:

```
input = [本轮 user message] // 仅这一条
instructions = systemPrompt + ledger("none") // ledger 显示无历史
```

模型完全不知道之前发生过什么。如果之前讨论的论文还是同一个、当前 Zotero item 也没换，**论文介绍（题录 + 摘要）会重新出现在 system prompt 里**——因为那部分来自 `buildSystemContextOnly(state.itemID, ...)`，由当前 itemID 决定，与 `state.messages` 无关。但任何**“我们刚才聊过 X” 的上下文都没了**。

3. 这两个按钮在整个上下文协议里的位置

回顾 `docs/CONTEXT_PROTOCOL.md` 的三层机制:

1. 对话文本（全量回放）
2. 历史 context 块（滑窗 + 字数预算 + 签名去重回放）
3. 上下文账本 ledger（拼到 system prompt）

这三层全部以 `state.messages` 为输入。所以:

- 「清空」直接砍掉所有三层的输入——后续请求三层全空;
- 「复制 MD」从三层之外旁路读取，输出到剪贴板，不进入任何 wire payload;
- 调试开关打开时，复制 MD 会临时调用 `formatContextLedger(state.messages)` 和 `buildSystemContextOnly(...)`，但**结果只塞进剪贴板字符串**，不会写回 state 或文件。

换句话说:「复制 MD」是**观察器 (observer)**，「清空」是**删除器 (destructor)**。两者作用面正交。

4. 其它易混按钮

工具栏里还有一些含 “清空” 字样的控件，**作用面完全不同**，不要混淆:

按钮	位置	作用
清空（聊天）	顶部工具栏，仅在 <code>messages.length > 0</code> 时显示	清空对话 + 删除持久化 thread
清空队列	任务队列面板， <code>sidebar.ts:1405</code>	清空“排队任务”列表， 不删除聊天内容
手动备份的“清空”操作	设置页 <code>hooks.ts:288</code>	清空导出文本框，不影响对话

5. 实践建议

1. **怕误删 → 先复制再清空**：「清空」没有撤销，没有“最近删除”。先复制 MD 备份对话，再清。
2. **想给模型瘦身但保留上下文 → 不要清空**：参考 `docs/CONTEXT_PROTOCOL.md` 第 5 节调 `policy.ts` 里的 `retainedContextChar-Budget`，让历史回放更克制；或者依赖模型自己调 `chat_get_previous_context` 工具按需重读。
3. **调试 wire payload 出现的内容 → 打开调试开关复制 MD**：这是当前最直接的“看模型实际看到了什么”的方式，比开浏览器开发者工具拦请求方便。
4. **持久化备份**：如果担心误删，可以在「设置 → 同步」里启用云同步，threads 会以可移植格式（`PortableThread`，见 `chat-history.ts:88-94`）周期写出，万一清空也能从远端恢复。

附：相关源码文件索引

- `src/modules/sidebar.ts:629-672` — 两个按钮的渲染与事件绑定
- `src/modules/sidebar.ts:3464-3465` — 调试开关 tooltip 文案
- `src/settings/chat-history.ts:113-142` — 持久化层（含空数组触发删除的关键逻辑）
- `src/context/message-format.ts:235-318` — `formatContextLedger`（被调试开关使用）
- `docs/CONTEXT_PROTOCOL.md` — 上下文协议主文档（每轮发什么 / 三层机制 / Codex 对照）